

# R-Control IP4 – WebSocket-Gateway (/ws)

---

## Was ist das?

Über WebSocket bleibt Ihre Anwendung dauerhaft mit dem Gerät verbunden und erfährt **sofort**, wenn sich etwas ändert — ohne regelmäßiges Nachfragen. Genau so arbeitet auch die Weboberfläche des Geräts.

Das ist die richtige Wahl für **Dashboards** und Visualisierungen. Wenn Sie nur gelegentlich etwas schalten oder abfragen, ist die [HTTP/JSON-API](#) einfacher.

---

## In 5 Minuten zum Ziel

Die Schnittstelle ist immer verfügbar und muss nicht eingeschaltet werden.

### Verbinden und zuhören — ein Beispiel im Browser

```
const ws = new WebSocket("ws://192.168.0.3/ws");

ws.onopen = () => console.log("verbunden");

ws.onmessage = (ereignis) => {
  const nachricht = JSON.parse(ereignis.data);
  console.log(nachricht.type, nachricht);
};
```

Direkt nach dem Verbinden schickt das Gerät einmalig ein **snapshot** mit dem kompletten aktuellen Zustand. Danach kommen laufend Aktualisierungen: **time** (im Sekundentakt, mit Uhrzeit und Temperatur), **relay** bei Schaltvorgängen, **button** bei Tastendrücken.

### Einen Ausgang schalten

```
ws.send(JSON.stringify({ type: "relay", index: 0, command: "on" }));
```

**index** ist **0-basiert**: 0 = Ausgang 1.

---

## Gut zu wissen

- **Höchstens 8 gleichzeitige Verbindungen.** Die Weboberfläche belegt selbst eine davon, sobald jemand sie geöffnet hat.
  - **Verbindungsabbrüche einplanen.** Bauen Sie einen automatischen Neuaufbau ein — nach einem Neustart des Geräts oder einer Netunterbrechung ist die Verbindung weg.
  - **wss:// bei aktiviertem HTTPS**, sonst **ws://**. Der Port ist derselbe wie der des Webservers.
-

Ab hier folgt der **vollständige Referenzteil** mit allen Details.

Diese Dokumentation beschreibt die WebSocket-Schnittstelle des R-Control IP4. Sie dient typischerweise für UI/Dashboard-Anwendungen, die Relais schalten und Status/Telemetrie in Echtzeit erhalten möchten.

## 1. Überblick

- **Pfad:** `/ws`
- **Protokoll:** WebSocket über HTTP oder HTTPS (abhängig von der Webserver-Konfiguration)
- **Format:** JSON-Nachrichten (Text-Frames)
- **Max. parallele Clients:** 8
- **Initialer Zustand:** Nach erfolgreichem Handshake sendet das Gerät einmalig ein `snapshot`.

## 2. Verbindungsaufbau

### 2.1 URL

- `ws://<gerät>:<port>/ws`
- `wss://<gerät>:<port>/ws`

Der Port entspricht dem HTTP/HTTPS-Port (siehe REST-API / Gerätekonfiguration).

### 2.2 Authentifizierung

Die WebSocket-Verbindung ist **token-geschützt**. Das Token kann auf zwei Arten übergeben werden:

1. **HTTP Header** (empfohlen für native Clients)

`Authorization: Bearer <token>`

2. **Query-Parameter** (praktisch für Browser-Clients)

`/ws?token=<token>`

#### Fehler bei fehlendem/ungültigem Token

- HTTP Status: `401 Unauthorized`
- Header: `WWW-Authenticate: Bearer realm="R-Control-IP4"`
- Body: Text `Unauthorized`

### 2.3 Token-Ablauf (Session)

- Das Gateway prüft die Token-Gültigkeit und verwaltet pro Verbindung eine Ablaufzeit.
- Wenn der Token abläuft, sendet das Gerät:

```
{"type": "session", "status": "expired"}
```

und schließt anschließend die Verbindung.

Hinweis: Im Gateway ist **kein Token-Refresh** über Response-Header wie bei REST implementiert. Clients sollten bei `session/expired` erneut einloggen und neu verbinden.

### 3. Kapazitätslimit

Bei zu vielen gleichzeitigen WebSocket-Clients wird der Handshake abgewiesen:

- HTTP Status: `503 Service Unavailable`
- Content-Type: `application/json`
- Body:

```
{"error": "ws_capacity", "message": "Too many dashboard clients connected, please try again later."}
```

### 4. Nachrichtenformat

- Jede Nachricht ist ein JSON-Objekt mit mindestens dem Feld `type`.
- Alle Message-Frames sind Text (`HTTPD_WS_TYPE_TEXT`).

### 5. Nachrichten vom Gerät (Server → Client)

#### 5.1 `snapshot` (Initialzustand)

Wird unmittelbar nach erfolgreichem Handshake an den neuen Client gesendet.

Beispiel (Struktur):

```
{
  "type": "snapshot",
  "relays": [
    {"index": 0, "state": false, "impulse_active": false},
    {"index": 1, "state": true, "impulse_active": true, "impulse_type": "on",
"impulse_remaining_ms": 12000}
  ],
  "epoch": 1739836800,
  "sun_location_valid": true,
  "next_sunrise_epoch": 1739865600,
  "next_sunset_epoch": 1739901600,
  "ntc_raw": 1234,
  "ntc_status": 0,
  "ntc_mv": 1500,
  "ntc_temp_c": 22.75,
  "scheduler": {"runtime_rule_count": 3, "runtime_max_rules": 64, "...": "..."},
  "actions": {"runtime_rule_count": 1, "runtime_max_rules": 64, "...": "..."}
}
```

#### 5.2 `time` (periodischer Zeit-/Sensor-Push)

Das Gerät sendet ungefähr **einmal pro Sekunde** ein **time**-Event.

Beispiel:

```
{
  "type": "time",
  "epoch": 1739836801,
  "sun_location_valid": true,
  "next_sunrise_epoch": 1739865600,
  "next_sunset_epoch": 1739901600,
  "ntc_raw": 1234,
  "ntc_status": 0,
  "ntc_mv": 1500,
  "ntc_temp_c": 22.75,
  "impulses": [
    {"index": 1, "type": "on", "remaining_ms": 11800}
  ]
}
```

### 5.3 relay (Relaiszustand geändert)

Wird bei Zustandsänderung eines Relais broadcastet:

```
{"type":"relay","index":1,"state":true}
```

### 5.4 impulse (Impuls fertig)

Wenn ein Impuls abgelaufen ist:

```
{"type":"impulse","index":1,"status":"done"}
```

### 5.5 ntc (NTC-Sensor Update)

Bei Sensor-Updates:

```
{"type":"ntc","ntc_raw":1234,"ntc_status":0,"ntc_mv":1500,"ntc_temp_c":22.75}
```

### 5.6 pong (Antwort auf Ping)

```
{"type":"pong","ts":123,"epoch":1739836801}
```

### 5.7 sched / sched\_ack

- `sched`: Scheduler-Konfiguration (inkl. Runtime-Metadaten) + Sonnenzeiten
- `sched_ack`: Status-Antwort nach `sched_set`

Beispiel `sched_ack`:

```
{"type": "sched_ack", "status": "ok"}
```

## 5.8 `actions` / `actions_ack`

- `actions`: Actions-Konfiguration (inkl. Runtime-Metadaten)
- `actions_ack`: Status-Antwort nach `actions_set`

Beispiel `actions_ack`:

```
{"type": "actions_ack", "status": "ok"}
```

## 6. Nachrichten an das Gerät (Client → Server)

### 6.1 `ping`

Client-Liveness/Roundtrip:

```
{"type": "ping", "ts": 123}
```

### 6.2 `relay_cmd`

Relais schalten / blinken:

#### Felder

- `index` (number): Relaisindex (0...3)
- `cmd` (string):
  - `on`, `off`, `toggle`
  - `blink_start` (+ `period_ms`)
  - `blink_start_adv` (+ `on_ms`, `off_ms`)
  - `blink_stop`

Beispiele:

```
{"type": "relay_cmd", "index": 0, "cmd": "toggle"}
```

```
{"type":"relay_cmd","index":2,"cmd":"blink_start","period_ms":500}
```

```
{"type":"relay_cmd","index":2,"cmd":"blink_start_adv","on_ms":100,"off_ms":900}
```

### 6.3 impulse

Impuls starten (Dauer wird intern auf **10...600000 ms** begrenzt):

```
{"type":"impulse","index":1,"state":"on","duration_ms":30000}
```

state: on oder off.

### 6.4 impulse\_cancel

Aktiven Impuls abbrechen:

```
{"type":"impulse_cancel","index":1}
```

### 6.5 time\_sync

Zeit setzen (epoch Sekunden):

```
{"type":"time_sync","epoch":1739836800}
```

Hinweise:

- Die Systemzeit wird gesetzt.
- In die RTC wird nur geschrieben, wenn epoch  $\geq$  2000-01-01 ist.
- Nach time\_sync sendet das Gerät sofort einen time-Push.

### 6.6 sched\_get / sched\_set

Scheduler lesen/schreiben.

- sched\_get löst ein sched-Broadcast aus.
- sched\_set erwartet:

```
{"type":"sched_set","data":{"... }}
```

Besonderheit:

- Wenn `data.astro` oder `data.location` vorhanden ist, wird daraus `config.system.astro` persistiert.

## 6.7 actions\_get / actions\_set

- `actions_get` löst ein `actions`-Broadcast aus.
- `actions_set` erwartet:

```
{"type":"actions_set","data":{" ... }}
```

- Das Gerät sendet danach `actions_ack` und anschließend ein aktualisiertes `actions`.

## 7. Beispiele

### 7.1 Browser (JavaScript)

```
const ws = new WebSocket(`ws://${location.host}/ws?token=${token}`);
ws.onmessage = (ev) => console.log(JSON.parse(ev.data));
ws.onopen = () => {
  ws.send(JSON.stringify({ type: "ping", ts: Date.now() }));
  ws.send(JSON.stringify({ type: "relay_cmd", index: 0, cmd: "toggle" }));
};
```

### 7.2 Python (websockets)

```
import asyncio, json
import websockets

async def main():
    uri = "ws://r-control-ip4.local/ws?token=" + TOKEN
    async with websockets.connect(uri) as ws:
        await ws.send(json.dumps({"type":"ping","ts":1}))
        async for msg in ws:
            print(json.loads(msg))

asyncio.run(main())
```

## 8. Sicherheitshinweise

- Der WebSocket-Handshake ist token-geschützt.
- Nach dem Handshake werden **keine zusätzlichen Permission-Checks** pro Message durchgeführt. Jeder erfolgreich authentifizierte Benutzer kann damit (Stand heute) auch schreibende Aktionen (Relais, Scheduler, Actions, Zeit) auslösen.
- Empfehlung: Tokens nur an Clients ausgeben, denen diese Steuerfunktionen explizit erlaubt sein sollen; ansonsten Rollen/Permissions serverseitig erweitern.

## 9. Troubleshooting

- **401 Unauthorized** beim Handshake: Token fehlt/ungültig.
- **503 ws\_capacity**: zu viele parallele WS-Clients; alte Sessions im Client sauber schließen.
- `{"type":"session","status":"expired"}`: Token abgelaufen → neu einloggen und neu verbinden.